

Discente: Gianni Kenyd Correa, Mestrado CAP.

Relatório de Execução de Código

O objetivo do código é navegar no repositório digital gerado pelo professor Renan Marujo, coletar links de artigos de uma edição específica de um evento e extrair metadados detalhados de cada documento, gerando ao final um arquivo CSV estruturado. Foram gerados dois códigos e em ambos houve o auxílio da IA Gemini para resolver os erros e auxiliar na implementação do código, um dos códigos gera o arquivo para 2014 e outro para 2003.

Bibliotecas Utilizadas

O código faz uso das seguintes bibliotecas utilizadas via Python:

- **playwright.sync_api:** É o motor principal da automação. Ele abre um navegador real (Chromium) e simula o comportamento de um usuário humano, permitindo navegação, espera pelo carregamento de elementos e a injeção de scripts na página.
- **pandas:** Utilizado para análise e estruturação de dados. No caso em questão foi usada para transformar a lista de dados em um formato de tabela, permitindo a exportação limpa para um arquivo CSV e a formatação da exibição no terminal.
- **time:** Biblioteca nativa do Python usada para controle de tempo. No código, é aplicada a função `time.sleep()` para impor pequenas pausas, garantindo que o navegador tenha tempo hábil para renderizar a interface visual antes da extração.

Acesso e Mapeamento Inicial

O Playwright é usado para iniciar o navegador e acessar a URL principal onde se encontra a lista de artigos do evento. Devido ao site utilizar uma estrutura de quadros `<frameset>`, o código isola especificamente o `frame[name="body"]`. Em seguida, ele aguarda o carregamento do primeiro link válido e executa um mapeamento, extraindo apenas os links absolutos `<href>` que contenham a tag identificadora de títulos `<x>`.

Navegação Individual

Com a lista de URLs em mãos, o código inicia um loop, visitando a página de cada artigo individualmente. O código utiliza uma breve pausa de segurança ao acessar o link direto do artigo.

Lógica de Extração

Dentro da página de cada artigo, o código executa uma função JavaScript que lida com a instabilidade de preenchimento dos metadados, onde ele opera com duas estratégias. Na primeira, O script varre o cabeçalho <head> do documento principal e de seus sub-frames em busca de metatags estruturadas. Se a informação estiver lá, ela é coletada. Caso o campo não tenha sido preenchido de forma oculta e retorne vazio, o script ativa a segunda estratégia, procura por rótulos visíveis na tela como "Resumo" e "Afiliação" em tabelas tags de negrito, capturando o texto seguinte, por fim, se ambas as estratégias falharem, o código interpreta que o dado realmente não existe.

Informações Extraídas

O código está estruturado para capturar e tabular os seguintes metadados de cada artigo, conforme o exemplo disponível no github:

- **Title (Título):** O nome do artigo científico.
- **Abstract (Resumo):** O texto de resumo do trabalho.
- **Tertiary Type (Tipo de Referência):** A classificação do documento (ex: Unidade Arquivística / Conference Proceedings).
- **Language (Idioma):** O idioma em que o trabalho foi redigido.
- **Conference Name (Nome do Evento):** O nome do simpósio ou conferência ao qual pertence.
- **Author (Autores):** O nome dos criadores do documento.
- **Affiliation (Afiliação):** A instituição, universidade ou organização à qual os autores pertencem.

Geração do Arquivo

Ao finalizar a leitura de todos os links, o pandas consolida os dados, cria o arquivo .csv, de modo que a exportação seja feita utilizando separadores ';', com codificação utf-8-sig, e por fim, uma gera uma representação visual da mesma tabela no terminal.